

Mini-projet Complexité — Structure de données « Arbres »

M1 Bioinformatique — Algorithmique avancée & Complexité

Sujet : Arbres n-aires ($n = 4$), représentations mémoire, opérations usuelles, complexités, évaluation expérimentale, applications bio-informatique

Date : 10/01/2026

Livrable exécutable (mono-fichier) : [projet_complexite.py](#)

1) Introduction

Les arbres sont des structures fondamentales pour modéliser des données hiérarchiques : systèmes de fichiers, taxonomies, arbres phylogénétiques, structures de décision, etc. Un arbre est un graphe orienté sans cycle muni d'une racine : il existe un unique chemin simple entre la racine et n'importe quel nœud.

Ce travail présente :

- des définitions essentielles (partie I),
- deux représentations mémoire (vecteur/contiguë et chaînée/dynamique) (partie II),
- l'implémentation d'opérations sur un arbre n-aire ($n = 4$) avec analyse de complexité (partie III),
- une évaluation expérimentale sur deux opérations (complet, sous-arbre complet maximal),
- des cas d'usage en bio-informatique (partie IV).

Conformément à la consigne « un seul fichier exécutable », l'implémentation complète (menu + représentations + opérations) est fournie dans [projet_complexite.py](#).

2) Objectifs du travail

- Rappeler les représentations mémoire d'un arbre (contiguë vs chaînée).
- Implémenter les opérations usuelles sur un arbre n-aire de mots (chaînes ≤ 20 caractères) avec leurs complexités en $\mathcal{O}(\cdot)$.
- Comparer (théorie vs expérimentation) le coût de certaines opérations.
- Illustrer l'usage des arbres en bio-informatique.

2.1 Utilisation du programme (format des entrées)

Le programme est piloté via un menu console. Quand une option demande une **information de nœud** (ex. racine d'un sous-arbre, nœud à rechercher, nœud à supprimer), il faut saisir **une seule étiquette** correspondant exactement à **info** dans l'arbre (ex. **A**, **B**, **C**, **A2**, **A21**).

Remarque : une saisie du type **11, 22, 17** (liste séparée par des virgules) n'est pas interprétée comme plusieurs nœuds ; elle est traitée comme une seule chaîne et sera donc « non trouvée » si aucun nœud ne porte exactement cette étiquette.

3) Partie I — Définitions

3.1 Arbre, nœud, racine

- **Nœud** : élément contenant une information (ex. une chaîne) + des liens vers d'autres nœuds.
- **Racine** : nœud unique sans parent.
- **Arbre** : ensemble de nœuds organisés hiérarchiquement (sans cycles), avec un parent unique par nœud (sauf la racine).

3.2 Degré, profondeur, hauteur

- **Degré (d'un arbre n-aire)** : nombre maximal d'enfants qu'un nœud peut avoir. Ici, $n = 4$.
- **Profondeur d'un nœud** : nombre d'arêtes entre la racine et ce nœud.
- **Hauteur d'un arbre** : longueur (en arêtes) du plus long chemin racine → feuille.
 - Dans notre implémentation : une feuille a une hauteur 0.

3.3 Chemin

Un **chemin** entre deux nœuds est une suite de nœuds reliés par des arêtes consécutives.

3.4 Arbre complet, dégénéré, équilibré

- **Arbre complet (n-aire)** : tous les niveaux sauf le dernier sont pleins (chaque nœud a n enfants), et le dernier niveau est rempli de gauche à droite.
- **Arbre dégénéré** : chaque nœud a au plus un enfant (comportement proche d'une liste chaînée), donc hauteur $\approx n_{\text{nœuds}} - 1$.
- **Arbre équilibré** : la hauteur est proche du minimum possible (les sous-arbres ont des tailles comparables).

3.5 Arbre binaire

Arbre dont chaque nœud a au plus 2 enfants : gauche/droit.

4) Partie II — Représentations d'un arbre en mémoire

4.1 Représentation chaînée (structure dynamique)

Principe : chaque nœud contient des références (pointeurs) vers ses enfants.

Dans ce projet, on utilise la représentation **premier fils / frère suivant** :

- **succ_gauche** : premier enfant,
- **succ_droit** : frère suivant,
- **parent** : lien vers le parent.

Avantages

- Flexible : insertion/suppression faciles.
- Pas besoin de taille max connue a priori.

Inconvénients

- Accès au k -ième enfant = parcours des frères : coût en $\$O(k)\$$.
- Surcoût mémoire par nœud (références).

4.2 Représentation contiguë (vecteur / tableau)

Principe : stocker tous les nœuds dans un tableau (liste), et représenter les liens via des indices.

Dans le projet, un nœud vecteur contient :

- **info** (chaîne),
- **parent** (indice, -1 si racine),
- **enfants** (tableau de taille **degre**, indices ou -1).

Avantages

- Accès direct aux enfants par indice (tableau fixe), coût $\$O(1)\$$ pour le slot.
- Compact si on utilise des indices simples.

Inconvénients

- Taille fixée par **degre**; on doit gérer les slots vides.
- Insertion/suppression peut être plus délicate si on veut compacter/réordonner.

4.3 Deux exemples (les deux représentations)

- **Chaînée** : construite via les fonctions d'opérations (arbre exemple 1 et 2).
- **Vecteur** : option menu « Exemples représentation vecteur » qui affiche une table (indice, parent, enfants).

5) Partie III — Opérations sur les arbres et complexité

5.1 Modèle et notations

On note :

- $\$n\$$: nombre de nœuds de l'arbre,
- $\$h\$$: hauteur,
- $\$k\$$: nombre d'enfants d'un nœud,
- $\$d\$$: degré maximal (ici 4).

5.2 Tableau comparatif (complexité théorique)

Opération	Idée / structure utilisée	Complexité temps	Complexité espace
Construire arbre (exemples)	ajout contrôlé par degré	$\$O(n)\$$	$\$O(n)\$$

Opération	Idée / structure utilisée	Complexité temps	Complexité espace
Affichage DFS	réursion premier fils / frère suivant	$O(n)$	$O(h)$
Affichage BFS	file (queue)	$O(n)$	$O(n)$
Compter nœuds	DFS récursif	$O(n)$	$O(h)$
Hauteur	DFS + max	$O(n)$	$O(h)$
Rechercher (info)	DFS	$O(n)$	$O(h)$
Chemin $a \rightarrow b$ (adresses)	remonter parents + LCA	$O(h)$	$O(h)$
Insérer sous b	recherche parent + ajout enfant	$O(n) + O(k)$	$O(h)$
Modifier nœud	recherche + affectation	$O(n)$	$O(h)$
Supprimer nœud	recherche + reconnection frères/enfants	$O(n) + O(k)$	$O(h)$
Afficher sous-arbre	DFS sur sous-arbre	$O(t)$	$O(h_t)$
Vérifier complet	BFS + invariant « trou »	$O(n)$	$O(n)$
Sous-arbre complet maximal	tester chaque racine candidate	$O(n^2)$ (implémentation actuelle)	$O(n)$
Extraire sous-arbre	copie DFS	$O(t)$	$O(t)$
Transformer en binaire	copie structure (gauche/droit)	$O(n)$	$O(n)$

Remarques :

- t = nombre de nœuds du sous-arbre extrait/affiché.
- Le « sous-arbre complet maximal » est implémenté simplement en testant la complétude pour chaque nœud : c'est correct mais coûteux.

5.3 Description algorithmique (résumé)

5.3.1 DFS (profondeur)

- Visiter le nœud.
- Parcourir récursivement ses enfants (chaîne via `succ_gauche` puis `succ_droit`).

5.3.2 BFS (largeur)

- Utiliser une file.
- Dépiler un nœud, empiler tous ses enfants.

5.3.3 Recherche

- DFS récursif : comparer `info`, sinon descendre dans les enfants.

5.3.4 Chemin entre deux nœuds

- Remonter depuis `a` vers la racine pour obtenir la liste des ancêtres.
- Remonter depuis `b` jusqu'à tomber sur un ancêtre de `a` : c'est le LCA.
- Construire le chemin $a \rightarrow \text{LCA}$ puis $\text{LCA} \rightarrow b$.

5.3.5 Insertion

- Rechercher le parent.
- Ajouter un enfant en fin de chaîne des frères (contrôle du degré).

5.3.6 Modification

- Rechercher le nœud.
- Remplacer `info`.

5.3.7 Suppression (version simplifiée mais robuste)

- Rechercher le nœud.
- Si feuille : l'enlever de la liste des enfants du parent.
- Si nœud interne : rattacher sa liste d'enfants au parent « à la place » du nœud supprimé.

5.3.8 Vérification arbre complet

- BFS niveau par niveau.
- Dès qu'on observe un nœud avec moins de `degre` enfants, on considère qu'un « trou » est apparu.
- Après ce trou, tous les nœuds visités doivent être des feuilles.

5.3.9 Sous-arbre complet maximal

- Énumérer tous les nœuds.
- Pour chaque nœud, tester si le sous-arbre enraciné ici est complet.
- Garder celui de plus grande taille.

5.3.10 Transformation en arbre binaire

- Construire un arbre binaire explicite `NoeudBinaire(info)`.
- Mapper `succ_gauche` → `gauche`, `succ_droit` → `droit`.

5.4 Exemples de problèmes NP-difficiles / NP-complets liés aux arbres

Remarque importante : beaucoup de problèmes deviennent **plus faciles** quand l'entrée est strictement un arbre (ex. vertex cover, dominating set, etc. sur les arbres se résolvent en temps polynomial). En revanche, de nombreux problèmes **impliquant** des arbres (comparaison de deux arbres, recherche d'un arbre optimal expliquant des données, etc.) sont NP-difficiles (souvent formulés NP-complets selon la variante).

Exemple 1 — Reconstruction phylogénétique optimale (Maximum Parsimony)

But : trouver l'arbre (topologie) minimisant le nombre total de changements d'états (caractères) sur l'arbre.

Pourquoi c'est difficile : explorer l'espace des topologies d'arbres possibles explose combinatoirement avec le nombre de taxons. Le problème d'optimisation est connu comme NP-difficile (souvent présenté comme NP-complet dans des formulations décisionnelles).

Conséquence pratique : on utilise des heuristiques (recherche locale, hill-climbing, branch and bound limité, etc.).

Exemple 2 — Sous-arbre commun maximal entre deux arbres (Maximum Common / Agreement Subtree)

But : étant donnés deux arbres (souvent non ordonnés), trouver un sous-arbre commun (mêmes étiquettes) de taille maximale.

Pourquoi c'est difficile : sur des arbres non ordonnés, l'alignement/association optimale des sous-structures peut devenir NP-difficile (et NP-complet dans certaines versions décisionnelles).

Lien bio-info : comparer des arbres phylogénétiques issus de méthodes différentes, ou mesurer la similarité de taxonomies.

6) Évaluation expérimentale

6.1 Méthodologie

- Génération d'arbres complets de tailles croissantes ($n = 10, 20, \dots$) via remplissage en largeur.
- Mesure du temps avec `perf_counter()`.
- Deux opérations mesurées :
 1. `est_arbre_complet`
 2. `sous_arbre_complet_maximal`

Pour limiter le bruit (processus en arrière-plan), chaque taille est mesurée **30 fois** et on reporte la **moyenne** et l'**écart-type**.

Environnement de test (machine de l'étudiant) :

- OS : Windows
- Python : 3.11.x
- CPU : Intel(R) Core(TM) i5-8350U CPU @ 1.70GHz
- RAM : ~7.86 GB

6.2 Résultats (moyenne $\pm$ écart-type, en secondes)

n	<code>est_arbre_complet</code>	<code>sous_arbre_complet_maximal</code>
10	0.000003 $\pm$ 0.000002	0.000022 $\pm$ 0.000008
20	0.000006 $\pm$ 0.000003	0.000052 $\pm$ 0.000021

n	est_arbre_complet	sous_arbre_complet_maximal
30	0.000012 ± 0.000006	0.000100 ± 0.000041
40	0.000024 ± 0.000006	0.000198 ± 0.000045
50	0.000027 ± 0.000011	0.000230 ± 0.000083
100	0.000032 ± 0.000013	0.000336 ± 0.000108
200	0.000119 ± 0.000048	0.001204 ± 0.000405
300	0.000116 ± 0.000063	0.001451 ± 0.000569
400	0.000230 ± 0.000087	0.002489 ± 0.000793
500	0.000323 ± 0.000077	0.003619 ± 0.000662

6.3 Interprétation

- Le temps de `est_arbre_complet` augmente globalement de façon proche de linéaire, cohérent avec $O(n)$.
- Le temps de `sous_arbre_complet_maximal` augmente plus vite : l'implémentation actuelle teste la complétude sur beaucoup de sous-arbres, ce qui se rapproche d'un comportement quadratique $O(n^2)$ (ce qui apparaît dans la croissance du temps).

7) Partie IV — Utilisation des arbres en bio-informatique

7.1 Arbres phylogénétiques

Besoin : représenter des relations évolutives entre espèces/séquences.

Données : taxons (feuilles), ancêtres (nœuds internes), longueurs de branches.

Opérations utiles

- Parcours / affichage (DFS/BFS) : $O(n)$.
- Recherche d'un taxon : $O(n)$ sans index.
- Calcul de distance entre deux espèces via LCA : $O(h)$ si parent/LCA.

Remarque complexité : certaines méthodes d'inférence (ex. maximum parcimonie ou maximum vraisemblance exact) peuvent devenir coûteuses (souvent NP-difficiles) sur des instances générales, d'où l'usage d'heuristiques.

7.2 Tries / arbres de préfixes pour k-mers

Besoin : indexer des motifs (k-mers) pour recherche rapide dans des collections de séquences.

Données : alphabet {A,C,G,T} (ou 20 AA), chaque chemin correspond à un préfixe.

Opérations

- Insertion d'un mot de longueur L : $O(L)$.

- Recherche d'un mot de longueur L : $\mathcal{O}(L)$.

7.3 Suffix tree / suffix trie (index de motifs)

Besoin : recherche de sous-chaînes dans une séquence longue (génomique).

Opérations

- Recherche d'un motif de longueur m : typiquement $\mathcal{O}(m)$ sur suffix tree.

7.4 Taxonomies et ontologies (ex. classification)

Besoin : représenter hiérarchies de fonctions/espèces.

Données : catégories (nœuds), relations parent→enfant.

Opérations

- Parcours, extraction d'un sous-arbre thématique : $\mathcal{O}(t)$.
- Recherche d'un terme : $\mathcal{O}(n)$ sans structure auxiliaire.

8) Conclusion

Ce mini-projet met en évidence l'intérêt des représentations mémoire des arbres :

- la représentation **chaînée** est flexible et naturelle pour les modifications,
- la représentation **contiguë** (vecteur) est adaptée aux accès indexés et à une structure plus « tableau ».

Les opérations de base (parcours, recherche, hauteur) sont en $\mathcal{O}(n)$, tandis que certaines opérations plus globales (comme la recherche naïve d'un sous-arbre complet maximal) peuvent devenir beaucoup plus coûteuses, ce que confirme l'évaluation expérimentale.

Annexe — Fichiers du projet

- **Livrable mono-fichier exécutable** : [projet_complexite.py](#)
- (Optionnel pendant le développement) : [main.py](#), [menu.py](#), [node.py](#), [operations.py](#)

Annexe — Captures d'écran (checklist)

À insérer dans le PDF (quelques captures significatives, sans redondance) :

1. **Menu principal** (affichage des choix)
2. **Construction d'un arbre** (exemple 1 ou 2) + affichage BFS aligné
3. **Affichage DFS** (preuve du parcours profondeur)
4. **Recherche** (nœud trouvé / non trouvé)
5. **Chemin entre deux nœuds** (affichage $a \rightarrow \dots \rightarrow b$)
6. **Insertion** (avant/après en BFS)
7. **Modification** (avant/après en BFS)
8. **Suppression** (avant/après en BFS)
9. **Vérification arbre complet** (résultat)

10. **Sous-arbre complet maximal** (racine + taille + affichage)
11. **Représentation vecteur** (tableau indices/parents/enfants)
12. **Évaluation expérimentale** (tableau des temps)
13. *(Optionnel)* **Affichage graphique tkinter**